

Vorwort

Ich bedanke mich bei Herrn Prof. Dr. Christian Jakob für die Betreuung der Arbeit auf einem Themengebiet, dass mich persönlich sehr interessiert und viel Freude bereitet. *Imir*

Ich bedanke mich bei der TH Aschaffenburg und allen beteiligten Mitarbeiterinnen und Professoren für die Organisation eines berufsbegleitenden Studiums. Ich schätze es sehr, dass es ermöglicht wird sich Chancen und Möglichkeiten zu erarbeiten, die man ohne den Studiengang nicht hätte. *1,*

Mitabeite Innen ?

Profenar Innen ?

1 Einleitung

Die Matrixmultiplikation findet in Gebieten Anwendung, die auf massive Datenverarbeitung und linearer Algebra basieren. Beispiele sind Neuronale Netze und Large Language Models (LLM) als eine Anwendung von neuronalen Netzen, sowie die 3D-Computergrafik. Dadurch sind sie ein extrem relevantes Leistungsmerkmal für moderne CPU-Architekturen.

massiver

In dieser Arbeit soll ein bestehendes RISC-V CPU Design um Anweisungen zur Berechnung der Matrixmultiplikation erweitert werden.

Zunächst wird vertieft dargestellt, weshalb die RISC-V-Architektur und die Matrixmultiplikation relevant sind. Dann wird die Aufgabenstellung an sich beschrieben. Es folgt eine Betrachtung des allgemeinen Entwicklungsstandes, in dem die Arbeit ausgeführt wird. Der Stand der Technik wird beschrieben. Das Kapitel wird mit einer Erläuterung zum Aufbau der Arbeit abgeschlossen.

1.1 Motivation

In dieser Arbeit soll eine bestehende RISC-V CPU um Anweisungen zur Durchführung von Matrixmultiplikationen erweitert werden.

RISC-V ist eine lizenzfreie Beschreibung eines Prozessors und der Anweisungen, die der Prozessor verarbeiten können muss. Durch diese Lizenzfreiheit ist die RISC-V-Architektur sehr interessant sowohl für den akademischen als auch für den kommerziellen Einsatz.

Die RISC-V-Architektur ist die fünfte Iteration des Berkeley RISC Projekts der Universität von Kalifornien. Die Arbeit an RISC-V begann im Jahre 2010 und ist inzwischen so ausgereift, dass seit einigen Jahren kommerzielle RISC-V Chips auf dem Markt verfügbar sind.

Die ersten verfügbaren Chips waren aufgrund der fehlenden Lizenzkosten und einfachen Umsetzbarkeit für wenige Cent verfügbar. Im Jahre 2025 gibt es RISC-V-Chips, die erweiterte Funktionen bieten, Desktop-Betriebssysteme und auch LLMs ausführen können.

Lizenzkosten

Die Firma Qualcomm beispielsweise als eines der weltweit führenden Unternehmen auf dem Gebiet der Halbleiter und der Telekommunikation hat eine RISC-V-Erweiterung namens XQC1 zu RISC-V und auch zu den Compiler-Framework LLVM beigesteuert [12]. Qualcomm führt auch Käufe von Firmen

Eigentlich muss immer ein Bindesteil kommen
wenn RISC-V - deutsches Wort.

1 Einleitung

mit RISC-V-Expertise durch. Dies deutet daraufhin, dass Qualcomm den Einsatz von RISC-V in Produkten ernsthaft in Erwägung zieht.

Die Matrixmultiplikation ist die Grundlage für perspektivische Projektionen, Rotationen und Bewegung von Objekten in der 3D-Computer-Grafik. Dort ist es notwendig 4x4 Matrizen zu verwenden und algebraische Berechnungen so schnell wie möglich auf diesen Matrizen ausführen zu können.

94

4. Transforms

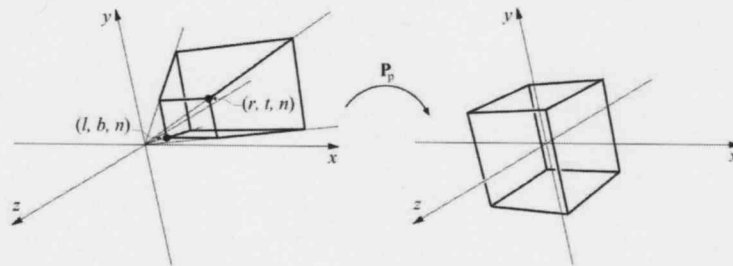


Abbildung 1.1: View Frustum

In Abbildung 4.1 aus [1] wird auf der linken Seite das Sichtfeld (View Frustum) einer Kamera in einer Szene dargestellt. Durch die Matrix P_p wird dieses Sichtfeld perspektivisch korrekt auf den Einheitswürfel auf der rechten Seite abgebildet. Durch das Umformen des Sichtfelds erscheinen weiter entfernte Objekte z.B. kleiner.

Diese Projektion muss auf jedes Objekt einer Szene angewendet werden. Mathematisch werden also viele Punkte mit der Projektionsmatrix multipliziert um die transformierten Punkte zu erhalten, die später auf einem Computerbildschirm erscheinen sollen. Berücksichtigt man die modernen 4K- oder 8K-Auflösungen und die hohen Bildwiederholraten von bis zu 200 Hz, wird deutlich, dass die Matrixmultiplikation extrem schnell und womöglich stark parallelisiert ablaufen muss um eine hohe Bildgenerierungsrate zu erreichen. Die Projektionsmatrix ist in Abbildung 1.2 aus [1] dargestellt.

$$P_p = \begin{pmatrix} \frac{2n}{r-l} & 0 & -\frac{r+l}{r-l} & 0 \\ 0 & \frac{2n}{t-b} & -\frac{t+b}{t-b} & 0 \\ 0 & 0 & \frac{f+n}{f-n} & -\frac{2fn}{f-n} \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

Abbildung 1.2: Perspective Projection Matrix

Der Einsatz von 3D-Computergrafik ist seit mehreren Jahren nicht mehr aus dem Alltag, aber auch aus vielen professionellen Bereichen wegzudenken. Man denke dabei an die Computerspiel- und Film-Industrie sowie Modellierungssoftware für Ingenieursanwendungen aller Art.

Seit Google im Jahr 2017 die Transformer-Architektur erfunden und daraufhin OpenAI ChatGPT 1 programmiert hat, setzt sich die Erfolgsgeschichte von Large Language Models bis heute fort. Generative künstliche Intelligenz wird als revolutionäre Erfindung betrachtet und führt schon jetzt dazu, dass sich Softwareentwickler Gedanken um ihren zukünftigen Wert auf dem Arbeitsmarkt machen. Es ist spätestens im Jahre 2026 klar, dass eine KI bestimmte Aufgaben bedeutend viel schneller erledigt, als es ein Mensch jemals könnte.

Firmen haben diesen Vorteil der KI erkannt und haben vor allem die hohen Lohnkosten menschlicher Arbeit im Blick. Hardware, die LLM berechnen kann, wird damit zum relevanten Posten in der Bilanz große Unternehmen. Anstatt fortlaufende Personalkosten zu bezahlen, möchten Unternehmen KI-fähige Hardware einsetzen, die naiv betrachtet nur einmalige Anschaffungskosten verursacht.

Es soll verdeutlicht werden, welcher Zusammenhang zwischen der Matrixmultiplikation und Neuronalen Netzen bzw. LLMs besteht.

Ein LLM basiert auf Neuronalen Netzen. Zwei Neuronen in einem sehr einfachen Netz sind in Abbildung 1.3 aus [17] dargestellt. Dabei sind $w_{i,j}$ die Gewichte und b_i die Biases, deren Werte durch Training über einer Datenbasis bestimmt werden. Nach dem Training soll das Netz auf Eingaben x_i mit sinnvollen Ausgabewerten y_i reagieren.

Zusammen werden Gewichte und Biases als Parameter bezeichnet. Das kleinste frei verfügbare LLM verwendet 10 Millionen Parameter, große LLM verwenden Billionen von Parametern.

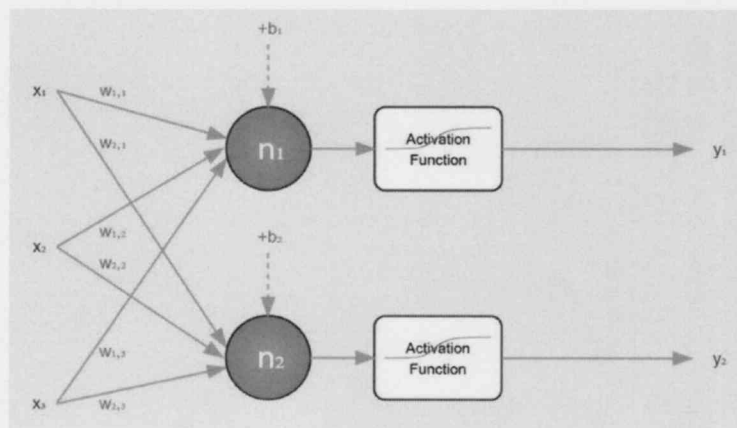


Abbildung 1.3: Einfaches Neuronales Netz

Ein Neuronales Netz lässt sich als Matrix beschreiben. Eingaben an ein Neuronales Netz erfolgen in Form von Werten $x_{i,j}$. Werden mehrere Eingaben x_i gleichzeitig an das neuronale Netz gegeben, wird aus den einzelnen Eingabevektoren eine Matrix. Dieser Umstand ist in Abbildung 1.4 aus [17] dargestellt.

$$\begin{bmatrix} w_{1,1} & w_{1,2} & w_{1,3} \\ w_{2,1} & w_{2,2} & w_{2,3} \end{bmatrix} \begin{bmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \\ x_3^1 & x_3^2 \end{bmatrix} = \begin{bmatrix} w_{1,1} \cdot x_1^1 + w_{1,2} \cdot x_2^1 + w_{1,3} \cdot x_3^1 & w_{1,1} \cdot x_1^2 + w_{1,2} \cdot x_2^2 + w_{1,3} \cdot x_3^2 \\ w_{2,1} \cdot x_1^1 + w_{2,2} \cdot x_2^1 + w_{2,3} \cdot x_3^1 & w_{2,1} \cdot x_1^2 + w_{2,2} \cdot x_2^2 + w_{2,3} \cdot x_3^2 \end{bmatrix}$$

Abbildung 1.4: Mehrere kombinierte Eingaben

Die rasante Verbreitung von LLMs hat Auswirkungen auf die Hardwarehersteller und den Handel der Hardware. Die Firma Micron hat beispielsweise einen Rückzug aus dem Endkundengeschäft vollzogen und verkauft RAM-Bausteine nur noch an Firmenkunden, die den RAM-Speicher dann bevorzugt für die Berechnung in Rechenzentren einsetzen. Es liegt nahe, dass LLMs in den Rechenzentren trainiert und betrieben werden. Die RAM-Preise für Endkunden sind aus diesem Grund innerhalb des Jahres 2025 um ein Vielfaches angestiegen.

Gleiches gilt für Grafikbeschleuniger der Firma NVIDIA. Auch hier konkuriert der Endkunde mit dem Firmenkunden. NVIDIA kann die Preise beinahe beliebig festlegen. Das Top-Produkt für Endkunden der Firma NVIDIA ist eine 5090 RTX Grafikkarte. Sie ist im Januar 2026 für etwa 3600 Euro erhältlich und kostet damit mehr als der gesamte Rest eines Computers. Eine Grafikkarte ist in der Lage Matrixmultiplikationen in großer Parallelität und extrem schnell zu berechnen. Grafikkarten werden auch zum Training von LLMs verwendet.

Getrieben durch den Erfolg von LLM kann sich eine Spezifikation für CPUs, wie z.B. RISC-V nicht mehr durchsetzen, wenn Sie keine Anweisungen für den effizienten Umgang mit Vektoren und Matrizen besitzt. Die Endkunden werden Produkte, die nicht in der Lage sind Matrizen hardwarebeschleunigt zu berechnen nicht mehr kaufen z.B. aus Gründen der Computergrafik. Das gleiche gilt für Firmenkunden, die den Betrieb eines LLM planen. Ohne beschleunigte Matrixberechnung kann sich eine CPU nicht mehr auf dem Markt behaupten.

Die CPUs der Firmen AMD und Intel besitzen bereits die Erweiterungen AVX2 (Advanced Vector Extensions 2), AMX (Advanced Matrix Extensions) und FMA (Fused Multiply Add) zum Rechnen mit Vektoren und Matrizen.

All diese Punkte zusammengefasst ergeben eine große Motivation für den Einsatz von RISC-V und den Einsatz von Matrixmultiplikation innerhalb einer RISC-V CPU.

1.2 Aufgabenstellung

Die NEORV32 RISC-V CPU wird um Anweisungen erweitert, die die hardwarebeschleunigte Matrixmultiplikation ermöglichen. Dabei orientieren sich die Anweisungen von der Kleinteiligkeit her an den Anweisungen, wie sie bereits für die RVV-Vektor-Extension ratifiziert worden sind. Im speziellen wird das Konzept des Strip-Mining ermöglicht. Dabei handelt es sich um die Auftei-

lung des Problems in Teile, die die CPU, aufgrund von begrenzter Ressourcen, schrittweise zugeführt werden, bis die Aufgabe als ganzes gelöst ist.

Die Erweiterung soll mit dem Wissenstand eines Beginners in der Hardware-Beschreibung vorgenommen werden können. Es wird im speziellen eine Erweiterung des bereits existierenden NEORV32 Core vorgenommen.

Die Matrizenberechnungen erfolgen im ersten Schritt auf Matrizen, deren Dimensionen ein Vielfaches der Zahl vier sind (also z.B. 4x4, 8x8, 16x16) und es werden nur die ganzzahlige Multiplikationen ohne Vorzeichen durchgeführt. Ohne Beschränkung der Allgemeinheit lässt sich eine Matrix an den Rändern mit Nullen auffüllen um auf ein Vielfaches der Dimension vier zu kommen, ohne das Ergebnis der Multiplikation zu verändern.

Da es viele Ansätze gibt Matrizen zu multiplizieren, wurde eine Einschränkung getroffen. Die Multiplikation erfolgt mit dem Outer-Produkt/Ansatz, da dieser Ansatz auch in dem maßgebenden Dokument zu dieser Arbeit beschrieben ist [11].

Handwritten notes on the right margin:

- 4 der 1 n
- 1 6
- 2 ns 15
- 1 -
- 1,
- Len
- 1 -

1.3 Stand der Technik

Ein RISC-V Komitee hat im Jahre 2015 mit der ersten Version der RISC-V Vector Extension (RVV) begonnen. Im Jahre 2021 wurde das Release 1.0 ratifiziert und ist jetzt in einen finalen Zustand (Frozen) überführt worden. RISC-V wird durch die (RVV) um Register für Vektoren und explizite Anweisungen zur Manipulation dieser Vektoren erweitert.

In der RISC-V Vector Extension Spezifikation [2] wird das Konzept des Strip-Mining beschrieben und hervorgehoben. Eine grafische Darstellung ist in Abbildung 2.3 aus [19] zu sehen.

Handwritten notes on the right margin:

- -
- (Klamme weg)

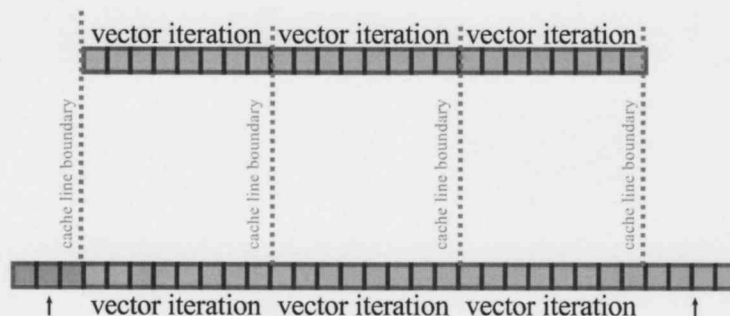


Abbildung 1.5: Strip Mining

Die Software lädt den Cache mit einem Teil einer oder mehrerer Vektoren voll und beginnt den Cache dann abzuarbeiten, in dem der Teil der Vektoren dann wiederum teilweise in den CPU geladen, verarbeitet und danach wieder aus dem CPU zurück in den Speicher übertragen wird. In Abbildung 2.3 wird einer dieser Schritte als "vector iteration" bezeichnet.

Handwritten note on the right margin:

- indem

Das Strip-Mining wiederholt diese Einzelschritte, bis die Gesamtaufgabe erledigt ist, also alle Elemente der Vektoren bearbeitet sind. Das Beispiel in Kapitel 6.4. namens "Example of stripmining and changes to SEW" aus [2] verdeutlicht dieses Vorgehen.

Der Vorteil der RVV Register ist es, dass diese Vektorregister mehrere Elemente parallel auf eine Instruktion hin zusammenrechnen können, anstatt jedes Element einzeln zu berechnen. Der CPU verfährt dann in einem echten SIMD-Betriebsmodus (Single Instruction Multiple Data) und spart ein Vielfaches an Taktzyklen im Vergleich zur sequentiellen Ausführung.

Das Prinzip des Strip-Mining wird auch in mathematischen Bibliotheken verwendet. Eine bekannte Bibliothek ist blis [20]. blis verwendet den Begriff Kernel oder Microkernel. Ein Kernel ist Code, der auf einem Cache-Frame ausgeführt wird, also den Code darstellt, den die CPU sehr schnell ausführen kann. Optimierte Anweisung für verschiedene CPU Architekturen werden hauptsächlich in dem Code verwendet, der dem Kernel zuzurechnen ist um Optimierungen dort durchzuführen, wo sie den meisten Nutzen bringen.

Um die Berechnungen im Kernel herum gibt es weiteren Code, der über Schleifen nacheinander alle Daten in den Kernel lädt, bis die Berechnung vollständig abgeschlossen ist.

Es gibt sehr starke Parallelen zwischen dem Strip-Mining und dem Kernel-Ansatz der blis Bibliothek. Sollte blis für RISC-V portiert werden, ist es sinnvoll, dass RISC-V das Prinzip des Strip-Mining ermöglicht. Dies wurde durch die RISC-V Vector Extension für Vektoren bereits erreicht.

Für die RISC-V-Matrixberechnung gibt es zum jetzigen Zeitpunkt keine ratifizierte Extension. Innerhalb der RISC-V-Arbeitsgruppen werden unterschiedliche Ansätze diskutiert. Die Ansätze werden im Kapitel zu den theoretischen Grundlagen beschrieben.

Der Status-Quo in der kommerziellen Entwicklung von RISC-V-Chips ist, dass bisher keine Chips mit Matrix-Erweiterungen produziert worden sind, da die Matrix-Spezifikation noch nicht existiert.

Einzelne Firmen haben allerdings mit der Entwicklung eigener Designs für die Matrixberechnung begonnen. Zu nennen ist die chinesische Firma Stream Computing die sich zur Mission gesetzt hat, high-end RISC-V Chips zum Einsatz in Rechenzentren zu entwickeln.

Bereits im Jahre 2022 konnte Stream Computing ein eigenes Arbeitsergebnis vorweisen [7]. Dieses Arbeitsergebnis wurde von Stream Computing selbst "RISC-V Matrix ISA Specification" in der Version v0.1 genannt. Die Version v0.5 wurde im Oktober 2024 erstellt. Es handelt sich aber trotz des Namens nicht um die offizielle Version, die das RISC-V-Gremium veröffentlichten wird. Stream Computing hat ebenfalls die notwendigen Anpassungen zum LLVM Compiler-Framework beigetragen. Die Änderung des LLVM Framework stellt eine große Investition in Form von Arbeitsstunden dar, die

Le

LV-
1,
IV

1 ein
Lgen
LV-
hif

LS-

-

-

1, -

-

Stream Computing aufgebracht hat ohne zu wissen ob ihre Ergebnisse jemals in der Breite zum tragen kommen werden.

Wie tragbar die Ergebnisse sind lässt sich schwer einschätzen ohne eine eingehende Evaluierung des Quellcodes durchgeführt zu haben. Es ist jedoch interessant zu sehen, welche Innovationskraft, Arbeitsgeschwindigkeit und auch welchen Mut und welche Risikobereitschaft die chinesischen Industrie an den Tag legt.

Innerhalb des NEORV32 CPU Umfeldes gibt es eine Arbeit von Qizal Aarsalan [3] bei der die NEORV32 CPU um Matrixberechnungen erweitert wird. Diese Bachelorarbeit unterscheidet sich allerdings hinsichtlich der Herangehensweise stark von Qizal Aarsalans Arbeit.

Die NEORV32 CPU besitzt im Design vorgesehene Erweiterungspunkte namens CFS und CFU. Das CFS (Custom Functions Subsystem) erlaubt es Hardware-Beschleuniger in den NEORV32 CPU einzubringen, die über Speicheradressen (Memory Mapping) aus einer Softwareanwendung mit Daten versorgt werden können. CFU (Custom Function Unit) erlaubt es einzelne Instruktionen zu ergänzen um den CPU um kleinschrittige Operationen zu erweitern.

Qizal Aarsalan untersucht den Ansatz der Matrixmultiplikation mit CFS und CFU. In dieser Bachelorarbeit werden Matrix-Instruktionen jedoch in den NEORV32 Chip eingebracht, die nicht über das CFS oder CFU Feature eingebunden werden sondern über die Execution-Engine, also der State-Machine/ des NEORV32 Chips. Der Grund ist, dass die Speicheradressabhängigkeit von CFS und CFU nicht konform mit einer Matrix-Extension-Spezifikation des RISC-V Gremiums sein können und nicht sein werden.

Bei CFS und CFU handelt es sich um Systeme, die nicht Teil der RISC-V Spezifikation sind. Diese Bachelorarbeit versucht die Ergebnisse einer Matrix-Extension im RISC-V Umfeld zu antizipieren und kann daher CFS und CFU nicht verwenden.

1.4 Struktur der Arbeit

Die Matrixmultiplikation wird im Kapitel "Theoretische Grundlagen" beschrieben. Es wird verdeutlicht, dass sich die Matrixmultiplikation in Teile aufteilen und damit schrittweise abarbeiten lässt.

Danach werden die momentan ablaufenden Anstrengungen des RISC-V Gremiums zur Erstellung einer Extension zur Matrixmultiplikation beschrieben. Dabei gibt es drei Ansätze. Der in dieser Bachelorarbeit untersuchte Ansatz wird zusammen mit den anderen Ansätzen vorgestellt. Dabei wird der Strip-Mining Ansatz erläutert und es wird eine Softwareanwendung besprochen, die diesen Ansatz als Computerprogramm implementiert.

1, 1,
IT,
Ld, Ln,
1p
LU-
wieso 2x
Qizal Aarsalan

L2-
1, 1p 1p
L2-

Lg-

Es folgt eine Beschreibung des Vorgehens bei der Erstellung dieser Bachelor-Arbeit.

Nachdem der Ansatz beschrieben ist, wird die NEORV32 CPU als das Ziel der Implementierung beschrieben. Das Design wird herausgearbeitet und es wird diskutiert, welche Änderungen an welchen Stellen vorgenommen werden, um die CPU um Matrixbefehle zu erweitern.

Danach wird der VHDL Quellcode der NEORV32 CPU erweitert und auf einen FPGA programmiert. Auf dem FPGA wird eine Software-Anwendung ausgeführt, die die Matrix-Instruktionen verwendet um eine Matrixmultiplikation durchzuführen.

Schließlich wird das Ergebnis mit einer normalen Matrixberechnung verglichen.

Die Arbeit endet mit einer Zusammenfassung und einem Ausblick auf mögliche Verbesserungen.

2 Theoretische Grundlagen

Dieses Kapitel beschreibt Vektoren und Matrizen aus der Mathematik und deren Berechnung durch ein Computersystem.

Bei der Berechnung ist auf die Geschwindigkeit zu achten. Ausgehend von dem naiven Berechnungsverfahren werden die Schwächen dieses Verfahrens diskutiert und beschrieben, welche Antworten das RISC-V Ökosystem auf diese Probleme hat.

Die ersten Hardwarebeschleunigungen sind mit der RISC-V Vector Extension für Vektoren eingeführt worden. Damit kann ein RISC-V Vektor SIMD unterstützen. Aufbauend auf der Vector-Extension wird diskutiert, welche Ansätze für die Berechnung von Matrizen verwendet werden können.

Der in dieser Arbeit gewählte Ansatz wird beschrieben.

2.1 Vektoren und Matrizen

Ein Vektor ist eine eindimensionale Gruppierung von skalaren Werten. Es handelt sich dabei um eine Fortführung von Skalaren zu neuen Objekten der Mathematik, wie sie auch z.B. bei komplexen Zahlen stattfindet.

Auf Vektoren sind mathematische Operationen wie Addition definiert, die wiederum Vektoren erzeugen. Es sind auch Operationen wie das Skalarprodukt oder der Betrag definiert, die aus Vektoren wieder Skalare erzeugen.

Eine Matrix wird in [18] als rechteckiges Zahlenschema mit m Zeilen und n Spalten beschrieben.

schema

= Einzahl

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} \\ a_{21} & a_{22} & a_{23} & \dots & a_{2n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & a_{m3} & \dots & a_{mn} \end{bmatrix}$$

Die Multiplikation zwischen zwei Matrizen A und B ist nicht kommutativ und nur definiert, wenn die inneren Dimensionen der Matrizen übereinstimmen, wenn also die Spaltenzahl der ersten Matrix mit der Zeilenzahl der zweiten Matrix übereinstimmt.

Zur Berechnung der Matrixmultiplikation



$$C = A * B \quad (2.1)$$

muss zur Berechnung einer Zelle der Matrix C ein Skalarprodukt aus einer Zeile von A und einer Spalte von B berechnet werden. Dabei wird zunächst aus der Spalte von B durch Transponieren eine Zeile gemacht und dann der Zeilenvektor aus A mit dem Zeilenvektor aus B durch das Skalarprodukt in einen Skalar umgerechnet. Dieser Skalar wird dann in die Matrix C eingesetzt.

Ein anschaulichere Beschreibung bietet das Falksche Schema [6].

2.2 Matrixmultiplikation mit SISD

Eine Implementierung, die sich direkt an der mathematischen Rechenvorschrift orientiert, verwendet drei For-Schleifen.

```

1 C = zero_matrix() // Starte mit Nullwerten
2
3 for int i = 1 to l // ueber die Zeilen von C
4   for int k = 1 to n // ueber die Spalten von C
5     for int j = 1 to m // ueber die Spalten von A
6       // und Zeilen von B
7       C(i,k) = C(i,k) + A(i,j) * B(j,k)
8     end
9   end
10 end

```

Der Nachteil dieser Beschreibung ist nicht sichtbar, wenn man sich den Pseudo-Code oder eine Implementierung in einer realen, höheren Programmiersprache ansieht. Diese Implementierung beschränkt sich auf SISD (Single Instruction Single Data)-Anweisungen. Wenn der Compiler das Optimierungspotential nicht erkennt, generiert er SISD-Anweisungen, die jede Multiplikation und Addition einzeln ausführen. Im aller ungünstigsten Fall besteht der generierte Code aus einer langen Folge aus einer Load-Anweisung gefolgt von einer Operation gefolgt von einer Store-Anweisung. Dieses ständige Laden und Speichern führt zu einer langen Laufzeit.

In den beiden Architekturen Van-Neumann und Harvard gilt gleichermaßen, dass eine Berechnung nicht im Speicher sondern nur innerhalb der CPU ausgeführt wird. Da die Daten im Speicher stehen muss also immer zunächst eine Übertragung zwischen Speicher und CPU erfolgen. Danach kann die Berechnung innerhalb der CPU durchgeführt werden und abschließen muss das Ergebnis zurück in den Speicher übertragen werden. Man nennt dies die Load-Store-Architektur. Dabei sind die beiden Schritte Load und Store aufwändig, da sie im schlimmsten Fall durch die gesamte Speicherhierarchie kaskadieren, wenn die Daten nicht im Cache vorliegen.

Der Nachteil der naiven Implementierung liegt also in der Tatsache begründet, dass Werte nicht in den Registern der CPU verbleiben können, da eine